The bug that was not a bug

A scheduled background task died on every run with `Service not found: @opencode/Skill`. The name said "skill". The fix was nothing to do with skills.

I had built the background-task system over the previous weeks: a task table, a runner that polls two queues, a generator agent that writes a task's spec, hooks, provenance, and a UI. The last piece — *actually running the task's prompt* — failed every single time. The task would be picked up, marked running, counted, and then die with that one line.

I spent an hour on the symptom. I added the skill service to the runner's dependencies. It still failed, one service further out: the session memory integration. I added that too. Then the formatter. Then ripgrep. Each fix moved the error to the next service in the chain, which is exactly what a wrong diagnosis looks like: the error is a symptom, and I was treating it as the cause.

What the error was actually about

DiveeOI is built on Effect-TS. Every service is a layer, and a layer only ever sees the services it declares as dependencies. That is the whole contract, and it is a good one.

The task runner was its own layer, so its context was its own declared dependencies — a dozen services. The HTTP route handlers, by contrast, ran with the entire application context: every service in the app graph, merged. The prompt path needs services *at call time* — the tool registry reads the skill service, the search tools read ripgrep, the patch tool reads the formatter — and those requirements propagate outward to whoever invoked the call. A route handler could satisfy them. The runner could not.

So the runner was calling into a code path that needed forty services while holding twelve. The error named whichever service happened to be resolved first.

The fix

One place now owns the list of application services. The HTTP graph is built from it, and the in-server workers — the task runner and the task generator — depend on it too, so a worker has exactly the context a request handler has. One list, two consumers, and the class of bug cannot come back.

The change was small. The reasoning was the work: I had to understand that `buildLayer` assembles a group by merging its members, that a member's implementation is provided with only its own dependencies, and that the fix therefore had to give the worker the merged set rather than more individual declarations.

What else this version fixed

- **Tasks that run, on a schedule, unattended.** The task carries a cron expression, an agent, a model, a directory, hooks, and its own session. The runner polls; a call journal makes a crash recoverable — committed calls replay verbatim, a call that was in flight surfaces as *interrupted, verify before retrying*.
- **The generator agent.** Give a task a one-line request and the generator writes the whole spec — prompt, schedule, agent, models, hooks — then stops, and the task's own agent runs it. The agent drives the same machinery from inside a session, through five tools: create, list, status, logs, control.
- **A router that learns.** DiveeComposer is not a model; it is a router that resolves each turn to a free model that is actually reachable. It prefers models that recently worked over models that merely look good in the catalogue, and it leaves a model alone for a day when it errors or answers with nothing — a ban that is written to the database, so a restart does not send the user straight back to it.
- **An agent that knows what it is.** The system prompt now says DiveeOI, in DiveeOI's words, and the built-in configuration skill was rewritten from "customize-opencode" to "customize-diveeoi" — because a model that thinks it is something else will write the wrong thing.
- **The film and the brand.** The mark, the wordmark, the palette and the motion language are now defined once and shared by the app and the two films, and the films' UI is a recreation of the real screens rather than a plausible invention.

Lessons I keep relearning

1. **A layer sees only what it declares.** When a service is "not found", ask what the caller holds, not what the message names.
2. **A truncated error is a hidden error.** The cause was sliced to 500 characters, which cut off every frame that said which service was missing. Raising the slice to 4 000 found the bug in one run.
3. **The catalogue is a prior, not the truth.** A gateway can list a model as free and then refuse it for want of a key. The router has to learn from refusals, not from the JSON.
4. **Evidence beats priors.** "It worked ten minutes ago" is worth more than "it is the biggest model available".
5. **Requirements propagate outward.** A tool that reads a service at call time pushes that requirement onto everyone who calls the tool. Design for it or get bitten by it.
6. **Make the failure legible.** The same bug, in a test, in a log, and in a UI, costs three different amounts of time to find. The log was the worst of the three.
7. **Determinism is a feature.** One paused timeline, no clocks, no random: a frame is a pure function of *t*. That property is what makes a render reproducible and a bug findable.

How I fix things in this project

The pattern is the same every time, and it has five steps.

1. **Name the failure before touching code.** Not "the task is broken" but "the runner holds twelve services and calls a path that needs forty". The second sentence is fixable; the first one is not.
2. **Get the whole stack.** Then get the *shape* of it: which frames are mine, which are the runtime's, and what is the first line that mentions something I own.
3. **Build a way to see it.** For the films it was a probe that reports geometry and opacity at any timestamp; for the server it was a live task and a real run. If you cannot see the thing, the first task is to make it visible.
4. **Fix the cause, then make the class impossible.** The skill service was a patch. The shared application service set was the fix, and it means the next worker cannot hit the same wall.
5. **Leave a marker.** The code now carries a comment explaining why the services are shared, and the registry explains why the films are built from sub-compositions. Six months later, that comment is the only thing that will stop the mistake from being made again.

What I would do differently

1. **Write the failing case down before the fix.** One sentence: "a scheduled task dies with a service-not-found error". I did not have that sentence, which is why I spent the hour on symptoms. The sentence turns a hunt into a reading.
2. **Keep the whole cause, always.** The 500-character slice I had been reading was cutting off exactly the frames that named the service. It cost ten minutes to raise it and then find the bug in one run. The cheap rule: if an error message ends mid-sentence, the bug is in what you cannot see.
3. **Ask what the caller holds.** "Service not found" is a statement about the caller's context, not about the service. Reading the layer contract is faster than adding dependencies.
4. **Make the seam small enough to share.** The application service list is a good seam because it is a list, not a framework. Whenever two things need the same set, that set deserves a name.
5. **Build the eye first.** Before the film could be judged, it needed something that could describe a frame to me without eyes: geometry, opacity, the transform, the filter. A probe is cheaper than a render and far cheaper than a wrong opinion.
6. **Write down what the next person will get wrong.** The comment above the shared service list exists so that the next person does not "fix" the runner by adding one more dependency. That comment is worth more than the fix.

The uncomfortable part of this work is that the bug was never about skills. It was about architecture, and the architecture was fine — it was just not shared with the thing that needed it. That is the shape of most of the bugs I have spent time on here, and the fix is almost never more code. It is a smaller distance between two things that should have been the same thing all along.